cruXipher CTF

TECHNICAL CHALLENGE WRITEUPS

Team BreakingBad

Prepared 25 October 2026

23 October 2026, 7:30 PM IST - 25 October 2026, 7:30 PM IST

19 cruXipher challenge writeups

Evidence-led analysis of web security, cryptography, forensics, reverse engineering, exploitation and OSINT.

Table of Contents

1. Executive Summary 4

2. Team Overview 4

3. Evidence and Verification 4

4. Challenge Index 4

5. Technical Notes 5

6. Detailed Technical Writeups 6

6.1 boxy proxy . . . 6

6.2 Internal Affairs . . . 8

6.3 JYU . . . 10

6.4 PDF Prince . . . 11

6.5 Wait For It . . . 13

6.6 House Music . . . 14

6.7 Funny Image . . . 16

6.8 Rice Writing . . . 17

6.9 dropped-as-a-baby . . . 18

6.10 zebra . . . 20

6.11 what-did-you-compress . . . 21

6.12 puffer . . . 23

6.13 Give me your seed . . . 25

6.14 Find the PIN . . . 27

6.15 Paper CPU . . . 28

6.16 T2F Program . . . 29

6.17 mlg . . . 31

6.18 santas-nice-list . . . 33

6.19 Is This Really OSINT? . . . 34

7. Cross-Challenge Technical Observations 36

8. Lessons Learned 36

9. Conclusion 36

Appendix A. Source Inventory and Reproduction 36

Appendix B. Preserved Minecraft Screenshots 38

Appendix C. Verification Notes and Glossary 41

1. Executive Summary

This report covers 19 cruXipher challenges spanning web security, cryptography, forensics, reverse engineering, binary exploitation, OSINT and miscellaneous data-recovery tasks. The writeups explain the observations, reasoning and tools behind each solution, from GraphQL credential filtering and compromised certificate trust to terminal reconstruction, custom compression, image encodings, virtual machines and PUF modeling.

House Music, Paper CPU, Rice Writing, T2F Program and Is This Really OSINT? include recorded platform acceptance. For the remaining challenges, the individual writeups describe the recovered output and the checks available for it.

2. Team Overview

Team: BreakingBad

Member	Profile
St3kr_80	cruXipher profile
sc0d3r	cruXipher profile
5p1r1t	cruXipher profile
Janakivallabh	cruXipher profile

3. Evidence and Verification

The report draws on the challenge writeups, solvers and retained artifacts. Checks that can still be reproduced are identified in each section. The companion inventory records the source files and their SHA-256 hashes.

We checked the writeups against the retained scripts and artifacts. Where an original attachment is missing, the relevant section distinguishes the recorded result from a step that can still be reproduced locally.

Recorded acceptance means a supplied writeup quotes a platform or service result. It is distinct from independently replaying an exploit, extracting a flag again from an original attachment, or checking only a final arithmetic/encoding step. This audit used local checks and did not contact challenge services, submit flags, or follow external repository links. Historical URLs and ports are retained as evidence, with no claim about their current availability.

Secrets, session tokens, account passwords and signed attachment-download tokens are omitted from this report. Challenge flags and non-secret puzzle inputs are preserved verbatim in their technical sections. Commands reflect the supplied environments: Linux commands require Linux or an appropriate compatibility environment; source paths and missing dependencies are identified rather than implied to exist.

4. Challenge Index

The sections below are grouped by the techniques used. Known point values appear beside the corresponding challenge; dynamic values reflect the recorded snapshot.

Section	Challenge	Category
6.1	boxy proxy	Web / cryptography
6.2	Internal Affairs	Web
6.3	JYU	Forensics

Section	Challenge	Category
6.4	PDF Prince	Forensics
6.5	Wait For It	Forensics
6.6	House Music	Forensics / audio
6.7	Funny Image	Misc / image steganography
6.8	Rice Writing	Misc / image analysis
6.9	dropped-as-a-baby	Misc / image recovery
6.10	zebra	Misc / EXIF and QR reconstruction
6.11	what-did-you-compress	Misc / custom compression
6.12	puffer	Misc / hardware modeling
6.13	Give me your seed	Misc / Minecraft seed search
6.14	Find the PIN	Reverse engineering
6.15	Paper CPU	Reverse engineering
6.16	T2F Program	Reverse engineering
6.17	mlg	Pwn
6.18	santas-nice-list	OSINT
6.19	Is This Really OSINT?	OSINT / web / cryptography

5. Technical Notes

Flag-shaped intermediate values require care. In Dropped as a Baby, the red-channel LSB string supplies a key; the assembled ciphertext still needs decryption to reach the final answer.

Flag prefixes vary across the included challenges: `cruXipher{...}` and short `cruX{...}`. PDF Prince's suggested full-prefix alternative is unverified. Decoy flags are identified within the relevant analyses. No uniform prefix is imposed.

Other material discrepancies are documented where they affect interpretation: Paper CPU's note about rejected submissions conflicts with its first-try statement; the compression bulletin differs from the working decoder. The literal counts in JYU and Find the PIN were checked and agree with their source strings.

6. Detailed Technical Writeups

Web and cryptography

6.1 boxy proxy

Category: Web / cryptography
Points: 500 (dynamic)

The service at `https://proxy.cruxipher.crux-bphc.com/` advertised a custom proxy protocol. Its supplied CA certificate and HKDF hint suggested that registration involved an authenticated key exchange. The original investigation found that `POST /register-proxy` attempted an outbound connection even when the HTTP response was `500 {"success":false}`. The callback, rather than the HTTP body, carried the useful result.

Protocol investigation

Registering a public TCP listener exposed a plaintext destination line, recorded as `144.24.133.118:8053`, followed by four-byte big-endian length-prefixed messages: a greeting, an RSA public key, a raw 32-byte X25519 public key, and an RSA-PSS signature over that X25519 key. The expected reply was a CA-signed certificate, our X25519 public key, and its signature made with the certificate's private key.

The supplied `CN=Pre-established CA` certificate used an RSA modulus whose two factors were recorded in `ca_factors.json`. We can reconstruct the signing key because knowing both primes gives the private exponent:

```
d = pow(e, -1, (p - 1) * (q - 1))
```

The solver checks `p * q == pub.n` before constructing the RSA CRT parameters. The saved factors pass that check and reconstruct a working signing key. The original factorization was obtained from FactorDB.

Completing the handshake

The documented verifier accepted the self-signed CA certificate itself as the client certificate. The solver therefore verifies the server's RSA-PSS/SHA-256 signature, signs our own raw X25519 public key using the recovered CA key, and sends the three reply frames. Both peers derive the session key as follows:

```
shared = client_key.exchange(X25519PublicKey.from_public_bytes(server_pub))
aes_key = derive_aes_key(shared)
plaintext = AESGCM(aes_key).decrypt(msg[:12], msg[12:], None)
```

`derive_aes_key` uses HKDF-SHA-256, length 32, no salt, and `info=b"handshake encryption"`. The first 12 bytes of the final message are the GCM nonce. The recorded plaintext begins “Hello Bob!” and contains:

```
cruXipher{som4h4W_b4tt4r_th4n_eMud4ra}
```

From the working directory, the supplied solver can be invoked with a public endpoint forwarded to local TCP port 4444:

```
python boxy-proxy/solve_boxy_proxy.py --port 4444 \
  --public-host <tunnel-host> --public-port <tunnel-port>
```

The security failure is the recoverable trust-anchor key combined with acceptance of a CA as a leaf identity. X25519 and AES-GCM do not compensate for compromised authentication. The RSA key sent in the same callback establishes signature consistency; that check alone does not establish an external identity.

Step-by-step procedure

1. Establish what registration actually does

The initial request accepts a host and port, rather than a proxy URL or credentials:

```
POST /register-proxy
Content-Type: application/json

{"host":"example.com","port":443}
```

The HTTP 500 did not distinguish a rejected registration from a callback that had already started. The recorded delay of roughly ten seconds for an unreachable address showed that the application was attempting a connection. That observation justified moving the investigation to a listener we controlled.

2. Capture the callback before implementing the response

The original setup used a Pinggy tunnel forwarded to local port 4444:

```
ssh -p 443 -R0:localhost:4444 tcp@a.pinggy.io
```

Once the returned host and port were registered, the server sent its destination line and greeting. The greeting specified both the frame format and the required authentication exchange. The solver reads each four-byte header as `>I`, then loops until the declared number of bytes has arrived. A single `recv()` would be insufficient because TCP does not preserve application message boundaries.

3. Recover the signing identity

Load the PEM certificate, read its RSA public numbers, and load the two saved factors. Verify their product before calculating the inverse exponent. The solver constructs `dmp1`, `dmq1` and `iqmp` as well as `d`, supplying the CRT parameters required by the cryptography library. This recovers a usable signing key without changing the certificate sent to the verifier.

4. Match the server's cryptographic parameters exactly

The server's signature covers the raw X25519 public-key bytes, not their hex representation or a PEM wrapper. RSA-PSS uses SHA-256, MGF1-SHA-256 and maximum salt length. Our reply signs the same representation with the recovered CA private key. The reply order is certificate, X25519 public key, then signature; all three use the same length-prefix format.

5. Decrypt the application message

The X25519 exchange supplies the shared secret. HKDF derives 32 bytes using the exact context string `handshake encryption`; changing that string produces a different AES key. Split the encrypted frame into a 12-byte nonce and the remaining GCM ciphertext/tag, and decrypt with no additional authenticated data. A valid GCM tag confirms that the key exchange and message parsing agree with the server. The flag arrives here, independently of the registration endpoint's HTTP status.

Source files: boxy-proxy-writeup.md, boxy-proxy/README.md, boxy-proxy/solve_boxy_proxy.py, boxy-proxy/cert/ca_cert.pem, boxy-proxy/cert/ca_factors.json.

6.2 Internal Affairs

Category: Web

The employee portal at `https://internal-affairs.cruxipher.crux-bphc.com` asked why Crooxus Dynamics nearly went bankrupt in February. Inspection of the React bundle, `/assets/index-CtrHTARE.js`, led to GraphQL at `POST /graphql`. The initial session was a manager account, so access to sensitive user fields was restricted.

Schema introspection exposed `User.apiKey`, `User.secretToken`, compensation fields and `AuditEvent` records. The description of `Query.auditEvents` instructed clients to send `X-API-Key` for full access. More importantly, `UserFilter` accepted the sensitive credentials even though the corresponding output fields were restricted. Matching user results therefore acted as a blind prefix oracle.

Aliased batch queries in `oracle2.py` recovered an administrator's credentials one character at a time. The script is missing from the archive, so the exact extraction queries cannot be reproduced here. With the recovered API key, the following query exposed the audit trail:

```
{ auditEvents(limit: 1000) {
    id actor { username } targetUser { username }
    action previousSalary newSalary comment
} }
```

With the recovered API key in the request header, event `AUD-2431` showed a self-approved compensation change by `theo.james` at `2026-02-18T16:42:09Z`: salary increased from 105125 to 42000000. The comment contained:

```
cruXipher{the_rats_read_the_audit_logs}
```

The visible compensation trend supplies a useful consistency check: `(765919.921875 - 111312.5) * 64 = 41894875`, exactly the salary increase. The failure was authorization on field output without equivalent protection on filtering, followed by API-key access to the audit feed. The flag was captured but not submitted.

Investigation sequence

1. Find the API behind the employee portal

The front end was a React application, so its bundled JavaScript was a useful starting point for identifying API paths. The recorded search against the locally saved bundle was:

```
grep -oE '"/[a-zA-Z0-9_./?=&-]*"' app.js
```

The `/graphql` string led to schema introspection. This query enumerated type and field names:

```
{ __schema { types { name fields { name } } } }
```

The important distinction was between fields visible in the schema and fields readable under the current session. `apiKey` and `secretToken` were restricted outputs, but their presence as filter inputs created a separate route to learning their values.

2. Locate the salary anomaly using permitted data

Before escalating access, the compensation trend narrowed the investigation:

```
query {
  compensationTrend {
    date personalSalary averageSalary markedAdjustment
  }
}
```

The marked jump on 18 February pointed to one compensation update rather than a gradual change across the workforce. The audit event identifier and timestamp then provided a specific target for the privileged query.

3. Use matching results as a credential oracle

The preserved extraction extends a candidate prefix one character at a time and asks whether a user matches it. A matching account reveals that the candidate is correct even when the credential itself cannot be selected. GraphQL aliases allow multiple candidate tests in one request. The original `oracle2.py` is absent, so the precise filter operators and alias construction are not reproduced as executable queries.

4. Read the full audit event

The recovered administrator API key was sent in `X-API-Key`, enabling the full `auditEvents` result. Inspecting actor and target together established that the same account changed its own compensation. Comparing old and new salary values gave an increase of 41894875, and the company-average calculation independently matched it. The event comment supplied the flag; it was not inferred from the salary numbers.

Source files: combined-writings.md.

Forensics

6.3 JYU

Category: Forensics
Points: 500

The challenge supplied `secret.pcap` and the hint “a pcap file patented in 1936.” The capture contained approximately 308 loopback packets over 104 seconds, with a single TCP session from `127.0.0.1:56243` to `127.0.0.1:9999`.

Following the stream exposed terminal escape sequences, Vim's insert-mode indicator and a new buffer named `flag.txt`. The client-to-server one-byte payloads carried keystrokes; terminal query responses had to be excluded.

```
tshark -r secret.pcap -q -z follow,tcp,ascii,0
tshark -r secret.pcap -Y "tcp.srcport==56243 && tcp.len==1" \
  -T fields -e data.data
```

The keys cannot simply be concatenated. The initial `i` enters insert mode. Later, Escape returns to normal mode, `b` moves backward, and `cw` replaces the final word. Seven backspaces remove part of the replacement, and `:wq` saves the buffer. The preserved final text is:

```
jpgQcld.p?k1m{b.y,0pt{j4lygp3+
```

The 1936 clue points to the Dvorak keyboard layout. Mapping each captured Dvorak character to its physical key and then reading that key as QWERTY produces:

```
cruXipher{v1m_netw0rk_c4ptur3}
```

For example, captured `j`, `p`, `g`, `Q` map to `c`, `r`, `u`, `X`; captured `?`, `{`, and `+` map to `{`, `_`, and `}`. Applying the mapping to the preserved string confirms the result.

The intermediate buffers contain 21, 18 and 22 characters. The final line has 30 characters, consistent with Vim reporting 31 bytes written including the newline. The PCAP is not retained, so the packet-level replay cannot be repeated from this archive. This challenge illustrates why terminal editing state must be reconstructed before applying a substitution.

Replaying the terminal session

1. Establish the capture format and conversation

```
file secret.pcap
capinfos secret.pcap
tshark -r secret.pcap -q -z io,phs
```

The loopback format and small packet count made a single interactive session plausible. Following TCP stream 0 exposed terminal screen-clearing sequences and Vim's `-- INSERT --` indicator. These identified the application state more reliably than searching the packet bytes for a flag prefix.

2. Separate terminal replies from user input

The client port, 56243, identifies the direction containing keystrokes. The one-byte filter isolates the typed characters, but the stream also contains terminal responses such as `ESC[2;2R` and color-report sequences. Those are replies to terminal queries, not text intended for `flag.txt`.

The preserved editing sequence is:

```
i jpgQcld.p?k1m{b.y,prt <ESC>
b cw 0pt{jmfoy.p <BS><BS><BS><BS><BS><BS>
j4lygp3+ <ESC> :wq <CR>
```

3. Track the buffer after each editing operation

Operation	Resulting text	Length
Insert the first string	`jpgQcld.p?k1m{b.y,prt`	21
Escape, b, cw	`jpgQcld.p?k1m{b.y,`	18
Insert replacement	`jpgQcld.p?k1m{b.y,0pt{jmfoy.p`	29
Seven backspaces	`jpgQcld.p?k1m{b.y,0pt{`	22
Insert final characters	`jpgQcld.p?k1m{b.y,0pt{j4lygp3+`	30

Here `b` and `cw` execute in normal mode. Treating them as literal text would corrupt every subsequent step. The final `:wq` is also a command and is excluded from the file content.

4. Convert keyboard positions after reconstructing the text

The mapping must be applied to the saved buffer, after deletions and mode changes. Captured `jpgQ` becomes `cruX`; the shifted keys matter as well, with `?` mapping to `{`, `{` to `_`, and `+` to `}`. The resulting 30-character flag agrees with the recorded 31-byte save once the trailing newline is included.

Source files: JYU-WRITEUP.md.

6.4 PDF Prince

Category: Forensics

The artifact was an internal memo whose producer string read `SanitizerPro 2.1 - history purged`. The recorded investigation found three `%%EOF` markers at offsets 7828, 8984 and 9234, plus repeated definitions of objects 1, 3 and 4. These are consistent with incremental PDF updates: a current cross-reference can supersede an object without removing its previous bytes.

```
pdfinfo chall.pdf
grep -abo '%%EOF' chall.pdf
pdftoppm -r 150 -png chall.pdf page
```

Two apparent flags were decoys. An uncompressed stream contained `flag{nice_grep_but_not_the_flag}`. Text extraction also yielded `ctf{copy_paste_is_a_trap}`, while rendering showed dashes. The writeup attributes that disagreement to a misleading `ToUnicode` map and zero-width spaces.

The latest catalog's optional-content configuration listed object 12, a hidden `keyring` layer. Its marked-content block supplied the repeating XOR key `L7-b5acb9c6`. The first revision of object 4 retained an encrypted hex string. XORing those bytes with the archived key produced big-endian 16-bit values, beginning `5, 126, 119, 187, 196`.

Because the font used `/Identity-H` and `/CIDToGIDMap /Identity`, these values identified glyphs. Reading the embedded `GhostMono` outlines, rather than trusting the extraction map, gave the original message:

```
cruX{xr3f_r3m3mb3rs_wh4t_y0u_d3l3t3d}
```

The supplied hex, XOR key and glyph table reproduce that string locally; the original PDF and font are unavailable for independent outline inspection. The source also suggests a `cruXipher{...}` alternative conditionally. No acceptance evidence supports that rewrite, so the artifact's short `cruX` prefix is retained. The lesson is to inspect historical objects, hidden layers and glyph appearance together when assessing PDF sanitization.

Recovering the deleted content

1. Compare the document structure with its sanitization claim

```
pdfinfo chall.pdf
strings -a chall.pdf | grep -i flag
grep -abo '%%EOF' chall.pdf
grep -abo 'obj' chall.pdf
```

The metadata says history was purged, but three EOF markers and repeated object definitions show that older revisions still occupy bytes in the file. The first obvious flag string therefore cannot be trusted merely because `strings` finds it.

2. Inspect both the current page and its text map

Inflate the latest object 4 content stream and compare its text operators with a rendered page. Object 9's `ToUnicode` CMap supplies the extracted text, while the actual embedded glyphs determine appearance. This comparison separates the copy/paste decoy from visible content. ROT13 explains the memo text but does not recover the deleted message.

3. Read the hidden optional-content group

The latest catalog includes `/OCGs [12 0 R]` and places that group in `/OFF`. Inspecting object 12 and its marked-content block reveals `KEYRING / layer 7` and the XOR key. Hidden-layer visibility is a display setting; the content remains available in the file.

4. Decode the first revision of the page stream

The original object 4 contains this hex operand to `Tj`:

```
4C322D1C351663D939A7361B3753623E61D462A463484C3C2DE0356A63E039FB36473753624861FE6268637D4C352D4335F
C636A39C8363B37B0627761686293633D4C162D69352363E2
```

Convert hex to bytes, XOR with the repeating bytes of `L7-b5acb9c6`, then group the result into big-endian pairs. The recovered identifiers are:

```
5,126,119,187,196,87,126,11,183,157,126,11,130,11,130,152,
11,126,125,157,81,75,2,33,157,8,171,119,157,66,11,170,11,33,11,66,128
```

These values are not plaintext character codes. They fall within the embedded font's glyph range and must be interpreted through its outlines.

5. Read the embedded font rather than its misleading Unicode map

Inflate object 8's `FontFile2` stream to recover the TrueType font. With `/Identity-H` and an identity CID-to-glyph mapping, each recovered value selects the corresponding outline. The initial IDs `5,126,119,187,196` spell `cruX{`. Continuing through the glyph list recovers the full short-prefix flag. This last step is why ordinary PDF text extraction cannot solve the challenge by itself.

Source files: PDFprince-writeup.md.

6.5 Wait For It

Category: Forensics

The capture contained roughly 8800 packets over 66 seconds, mostly TCP and DNS. Two IRC sessions reached `10.13.37.7:6667`, from `192.168.1.23` and `192.168.1.42`. Their repeated messages included plausible decoy flags; the second conversation supplied the useful hint, “the flag is in the timing.”

The investigation focused on server messages in the first conversation. A recorded stream number was 2, but stream numbers depend on the capture and should be confirmed before reuse.

```
tcp.port == 6667
tcp.stream == 2
```

In Wireshark, “Seconds Since Previous Displayed Packet” made the timing pattern visible. Ordinary intra-burst gaps were approximately 6-14 ms. Larger pauses clustered around 0.1 s and 0.3 s. Bursts contained about 10-14 or 27-38 messages, giving a similar short/long ratio.

The recorded interpretation treats a short burst as a Morse dot and a long burst as a dash. Short pauses separate symbols; long pauses separate letters. Counting the groups yielded:

```
T1M1NG_1S_3V3RYTH1NG
```

The phrase began repeating before the capture ended. Wrapping the recovered payload gives:

```
cruXipher{T1M1NG_1S_3V3RYTH1NG}
```

The capture is missing, so the thresholds, treatment of underscores and full symbol sequence cannot be replayed here. The technical distinction is that message counts and inter-packet delays carry the payload while application text acts as cover traffic.

Timing analysis procedure

1. Identify the two IRC sessions

In Wireshark, open **Statistics > Conversations > TCP** and find the connections to `10.13.37.7:6667`. Follow each TCP stream before assigning it a role. The `192.168.1.23` conversation carries `#ctf`; the `192.168.1.42` conversation carries `#offtopic`. Their visible flags are decoys, but the latter conversation supplies the timing hint.

2. Isolate the relevant rhythm

Return to the `#ctf` conversation, confirm its stream number, and set the time display to **Seconds Since Previous Displayed Packet**. The recorded stream was 2. Examine the server's repeated message groups: the useful signal is the burst-and-pause pattern, rather than the changing text or total TCP stream size.

3. Classify bursts and pauses separately

Observation	Recorded range	Interpretation
Within-burst spacing	6-14 ms	Ordinary packet rhythm
Short burst	About 10-14 messages	Morse dot
Long burst	About 27-38 messages	Morse dash
Short pause	About 0.1 s	Continue the same letter
Long pause	About 0.3 s	Finish a letter

Both burst lengths and pause lengths have approximately a 1:3 ratio. That joint pattern makes Morse a stronger hypothesis than treating every packet delay as a raw binary bit.

4. Transcribe one complete repetition

Count each burst up to the following large pause, assign a dot or dash, and use the longer pauses to delimit letters. Near the start, a roughly 35-message burst followed by a long pause gives a single dash, `T`. The recorded complete payload is `T1M1NG_1S_3V3RYTH1NG`; the next repetition begins before the capture ends. The surviving writeup does not retain the full Morse transcription or underscore handling, so those details cannot be reconstructed further without the PCAP.

Source files: Waitforit-writeup.md.

6.6 House Music

Category: Forensics / audio

The input was `transmission.wav`, recorded as 16-bit stereo PCM at 44100 Hz with 352584 frames. Common noise obscured the useful signal in either channel alone. Subtracting right from left using signed 32-bit arithmetic cancelled the shared component without overflowing 16-bit sample values.

A spectrogram of the difference signal revealed 17 Pigpen glyphs. The clue “CRUX first” defines a keyed alphabet by prepending CRUX and then appending the remaining letters:

```
CRUXABDEFGHIJKLMNOPQSTVWYZ
```

The undotted grid was `CRU / XAB / DEF`, the dotted grid `GHI / JKL / MNO`, and the undotted cross top/left/right/bottom `P/Q/S/T`. The preserved zero-based indices were:

```
20,18,7,0,21,1,17,18,10,4,20,7,15,4,20,17,16
```

Indexing the keyed alphabet reproduces `SPECTROPHASEMASON`, which was wrapped as:

```
cruXipher{SPECTROPHASEMASON}
```

The recorded CTFd response for challenge 1 reports status `correct`. The WAV and plotted glyphs are no longer retained; the keyed-alphabet conversion remains reproducible. The practical lesson is to compare stereo channels algebraically before treating noise as irreducible, and to preserve generous plot margins when reading edge glyphs.

Audio recovery steps

1. Confirm that the file contains two usable channels

The original inspection used `file`, `sha256sum` and `exiftool` on `transmission.wav`. Its 352584 stereo frames at 44100 Hz represent about eight seconds of audio. The two channels are important: a single-channel conversion would discard the relationship used to suppress the cover signal.

2. Form the channel difference before plotting

Convert signed 16-bit samples to a wider signed integer type before computing left minus right. If both channels contain the same noise component `N`, then `(L + N) - (R + N)` cancels it. The retained notes identify subtraction, rather than listening to either channel alone, as the step that exposed the glyphs. They do not retain the exact spectrogram parameters, so no FFT size or plotting configuration is invented here.

3. Build the keyed Pigpen alphabet

Read the resulting 17 glyphs against the keyed arrangement. “CRUX first” means those four letters occupy the first four alphabet positions, with duplicates omitted when appending the remaining alphabet. The dotted and undotted shapes distinguish the two grids; direction distinguishes the cross positions. Using an ordinary A-Z Pigpen layout yields the wrong substitution.

4. Check the transcription independently of the plot

The preserved indices permit a compact check even though the audio and spectrogram are missing:

```
alphabet = 'CRUXABDEFGHIJKLMNOPQSTVWYZ'
indices = [20,18,7,0,21,1,17,18,10,4,20,7,15,4,20,17,16]
print(''.join(alphabet[i] for i in indices))
```

This prints `SPECTROPHASEMASON`. It validates the substitution after transcription, while the recorded CTFd acceptance supports the final wrapped result. The glyph-reading stage still depends on the original spectrogram.

Source files: combined-writings.md.

Miscellaneous

6.7 Funny Image

Category: Misc / image steganography
Points: 500

The challenge paired a nearly uniform RGBA image with clues about subtraction and reversed coordinates. The image contained 52 isolated red-channel pixels whose values were rare and differed from their surroundings by six brightness levels. Each occupied a separate 256 x 256 block, spanning 16 blocks across each of three rows and four blocks in a fourth row.

The supplied script selects red values occurring fewer than 100 times, finds their coordinates, and sorts points by `(y // 256, x // 256)`. The global coordinates establish reading order; the offsets within each block carry character values.

```
x_offsets = [px % 256 for px, py in points]
y_offsets = [py % 256 for px, py in points]
flag = ''.join(chr(px - py)
               for px, py in zip(x_offsets, reversed(y_offsets)))
```

Reversing the entire y-offset sequence, rather than each row separately, is essential. The first x offset is recorded as 129 and the final y offset as 30: `129 - 30 = 99`, or `c`. The following pairs recover the flag prefix and then the payload:

```
cruXipher{r_rAnDom_Keey_Th4T_i56_vErY_uNCrAcKaBl237}
```

With Pillow and NumPy installed, reproduction uses:

```
python "Funny Image/solve.py" funny-image.png
```

The original PNG is not retained with the solver. A resized preview cannot substitute for it because the decoding depends on exact pixel coordinates. This is a coordinate encoding, so preserving the original image bytes is necessary for future verification.

Pixel-decoding steps

1. Use the full-resolution red channel

The original image is 4096 x 4096 RGBA. The solver converts it to a NumPy array and selects channel 0. It does not average RGB values or inspect transparency. This matters because the anomaly selection is based on exact red-channel frequencies.

2. Isolate rare values, then preserve their coordinates

The following excerpt is from the retained solver:

```
values, counts = np.unique(pixels, return_counts=True)
y, x = np.where(np.isin(pixels, values[counts < 100]))
points = sorted(zip(x.tolist(), y.tolist()),
                key=lambda p: (p[1] // 256, p[0] // 256))
```

The frequency cutoff removes the large uniform regions and keeps sparse modified pixels. `np.where` returns row indices first, so assigning them to `y, x` avoids transposing the coordinate roles. Sorting by block row and column then recovers the intended spatial reading order.

3. Separate block placement from character data

Division by 256 locates the block; modulo 256 gives the encoded offset inside it. Only the offsets enter the subtraction. Using absolute coordinates would mix the layout grid into the character values and introduce multiples of 256.

4. Reverse the y sequence and subtract

The reversal applies to the entire ordered y-offset list. Pair the first x offset with the last y offset, then work inward. The first pair gives `129 - 30 = 99`, followed by the remaining characters. The script asserts the `cruXipher{` prefix and closing brace after decoding. Those format checks are useful sanity checks, but reproducing the result requires the original image rather than a screenshot of it.

Source files: Funny Image/writeup.md, Funny Image/solve.py.

6.8 Rice Writing

Category: Misc / image analysis
Points: 50 (recorded API value)

The input was a 10000 x 10000 RGB image containing 311 rice grains. The challenge's sorting, occupied-space and remainder clues pointed to connected-component area rather than visual identification of the grains.

The documented workflow thresholds the grayscale image at 120, measures each foreground component's pixel area, sorts those areas ascending, and converts `area % 256` to a character. Thresholds 100, 120, 140 and 160 reportedly produced the same 311 components and areas, supporting the segmentation choice.

The first recorded areas were `41026, 41330, 41593, 41825, 42094`. Their remainders decode to `Bryan`. The full 311-character plaintext reportedly included the substring:

```
othercruXipher{riCegOesWiThaNyThinG}personalities
```

The extracted flag was:

```
cruXipher{riCegOesWiThaNyThinG}
```

The notes quote a successful submission to challenge 9 and a follow-up API record with `solved_by_me: true`, `attempts: 1`, and `value: 50`. The original image, `areas.json` and `decode.py` are not supplied, so only the first-five-area arithmetic can be checked locally. The method demonstrates how component measurements can encode text even when the image looks like an ordinary object arrangement.

Extracting text from component areas

1. Segment the grains at several thresholds

The documented analysis tested grayscale thresholds 100, 120, 140 and 160. All four gave 311 foreground components with the same areas. That stability matters because an area-encoded message would be damaged by stray pixels, merged components or a threshold that clips grain boundaries.

2. Measure area rather than position or outline

Connected-component labeling groups touching foreground pixels into individual grains. For each grain, record the count of occupied pixels. The clue about the space it occupies points to this measurement; the bounding-box area would include background pixels and is not the quantity used in the solution.

3. Sort before converting measurements to bytes

The source's sorting clue establishes ascending area as the character order. Reading components in scan order would retain image placement instead and scramble the message. Reduce each sorted area modulo 256 to recover one byte per grain.

Area	Remainder modulo 256	Character
41026	66	B

Area	Remainder modulo 256	Character
41330	114	r
41593	121	y
41825	97	a
42094	110	n

The first five characters form `Bryan`, providing an early check that segmentation, ordering and arithmetic agree. The complete 311-character text contains the flag within a larger sentence, so recovery requires locating the flag substring rather than wrapping the entire message.

4. Preserve the decoded case

The mixed capitalization in `riCegOesWiThaNyThinG` comes from the measured bytes. Normalizing its case would change the answer. The recorded successful submission and `solved_by_me` response confirm that exact string; the image and full area list are missing from the present archive.

Source files: combined-writings.md.

6.9 dropped-as-a-baby

Category: Misc / image recovery
Points: 300

The archive contained fourteen PNG-named files with damaged signatures and IHDR fields. The red-channel LSB string provides a key hint; assembling the visual fragments and applying Vigenere decryption recover the final flag.

Repairing the image data

The solver concatenates IDAT payloads and inflates the resulting zlib stream. The actual scanlines use 8-bit RGB and filter type 0. Two decompressed lengths determine the geometry:

```
360400 = 400 * (1 + 300 * 3)  -> 300 x 400 pixels
360300 = 300 * (1 + 400 * 3)  -> 400 x 300 pixels
```

It asserts these lengths and filter bytes, writes a correct PNG signature and IHDR, recompresses IDAT and calculates new chunk CRCs. Portrait tiles are rotated; the archived layout applies flips and arranges twelve tiles into the ciphertext region.



Figure 1. Preserved assembled ciphertext image. The lowercase l after q1n matters to the transcription.

Separating the hint from the answer

Reading red-channel LSBs from `dat_0a62c9` in row-major order and packing each group of eight bits MSB-first revealed `cruXipher{k3y_1s_HEADACHE}`. The combined notes report that as the flag. The later solver extracts `HEADACHE` from it and applies the key to the assembled text:

```
jvuAiroiy{xh1v_1m4il_q1nl7_h4y3_4_c0pjyzw1oq}
```

The implementation preserves case, leaves digits and punctuation unchanged, and advances the key only on alphabetic characters. That exact transcription decrypts to the saved final `flag.txt`:

```
cruXipher{th1s_1m4ge_m1gh7_h4v3_4_c0ncuss1on}
```

The final decryption was checked locally against the saved flag. PNG repair and LSB extraction cannot be repeated without the original ZIP. To rerun the supplied pipeline when it is restored:

```
python dropped-as-a-baby/solve.py dropped-as-a-baby.zip
```

The solver contains a manually recovered tile layout and ciphertext transcription; it does not infer those automatically. The documentation lesson is to follow a key-shaped intermediate result through the remaining layers before treating it as the final answer.

Full repair and decryption procedure

1. Traverse the damaged PNGs without trusting IHDR

The solver begins chunk parsing at byte offset 8 and reads each chunk's big-endian length and four-byte type. It collects only IDAT data for decompression. Damaged signatures, incorrect header fields, ancillary decoys and trailing data therefore do not have to be accepted by a normal PNG viewer first.

2. Reconstruct valid scanlines and chunk checksums

Each row has one filter byte followed by three bytes per pixel. The solver selects one of the two supported dimensions from decompressed length and asserts that every filter byte is zero. It builds a fresh IHDR using bit depth 8 and color type 2, then calculates each new chunk CRC over its type plus payload. This repairs the container while retaining the decompressed RGB values that carry the hidden data.

3. Read the red-channel hint

The retained extraction is:

```
a = np.array(images['dat_0a62c9'])
hidden = np.packbits((a[:, :, 0].reshape(-1) & 1),
                     bitorder='big').tobytes()
```

Flattening in row-major order fixes pixel order. Taking `& 1` keeps each red value's low bit; MSB-first packing turns every eight pixels into a byte. The extracted `k3y_1s_HEADACHE` phrase tells us that this stage provides a decryption key.

4. Assemble the visible text

The solver rotates portrait pieces to landscape and applies the stored flips before placing twelve tiles in a three-by-four arrangement. The arrangement was recovered manually and is hard-coded. The saved ciphertext image supports the transcription, including the lowercase `l` after `q1n`. Substituting digit `1` there would change which characters consume the Vigenere key.

5. Decrypt while keeping key position aligned

For a letter, subtract the corresponding `HEADACHE` shift modulo 26. Preserve its original case and advance the key index by one. Copy digits, braces and underscores unchanged without advancing the index. The first nine letters decrypt to `cruXipher`, providing a check before accepting the rest of the text. The final result matches the retained `flag.txt` exactly.

Source files: dropped-as-a-baby/writeup.md, dropped-as-a-baby/solve.py, dropped-as-a-baby/flag.txt, dropped-as-a-baby/recovered-ciphertext.png, combined-writings.md.

6.10 zebra

Category: Misc / EXIF and QR reconstruction
Points: 285

The input, `buncharandoms.zip`, contained 841 uniformly black or white JPEGs, each 10 x 10 pixels. Since `841 = 29 * 29`, the images could represent QR modules. Random filenames concealed their order, but EXIF `UserComment` retained a one-based position.

The solver accesses IFD 34665 and tag 37510, skips the eight-byte ASCII marker, and concatenates digits from the comment. Examples include `34rot9ten -> 349` and `4enter66 -> 466`. It rejects duplicate positions and checks that the complete index set is a square numbered consecutively from 1.

```
x = (index - 1) % side
y = (index - 1) // side
```

Each image's pixel `(0, 0)` supplies its module color. A four-module white border and nearest-neighbor enlargement make the reconstructed code readable.



Figure 2. Preserved QR reconstruction from the source folder.

The recorded QR payload is:

```
Y3J1WGlwaGVyezFfbDB2M18zeDFmX20zdDRkYXQ0fQ==
```

Base64 decoding gives:

```
cruXipher{1_l0v3_3x1f_m3t4dat4}
```

The Base64 step was independently checked. The original ZIP is absent, preventing a new audit of all EXIF indices. Reproduction requires Pillow and OpenCV:

```
python zebra/solve.py buncharandoms.zip
```

The original category line says “Misc / compression / custom-format,” but the documented technique is EXIF ordering followed by QR and Base64 decoding. The index uses that more precise technical description while preserving the broad Misc classification.

Reconstructing the QR modules

1. Inspect the image set as a whole

The 841 tiny uniform JPEGs are better treated as modules of a larger image than as separate photographs. A square root gives a 29 x 29 layout. The filenames do not supply useful ordering, so metadata becomes the next source of structure.

2. Recover an index from each EXIF comment

The solver opens each ZIP member directly, retrieves `UserComment` from the EXIF IFD and removes its eight-byte encoding marker:

```
comment = im.getexif().get_ifd(34665)[37510]
text = comment[8:].decode('ascii')
index = int(''.join(re.findall(r'\d', text)))
```

Only decimal digits contribute to the index; the surrounding word is noise. The checks for duplicate indices, consecutive numbering and a square count prevent a partially reconstructed grid from being mistaken for a valid result.

3. Place each module in row order

Convert the one-based index to zero-based coordinates using `(index - 1) % side` and `(index - 1) // side`. The uniform color at pixel `(0,0)` supplies the module value. This avoids deriving module placement from ZIP order, which has no supported significance.

4. Add a quiet zone and decode both layers

The code is pasted into an image eight modules wider and taller, leaving four white modules on each edge. Nearest-neighbor scaling preserves hard module boundaries. OpenCV first decodes the QR payload; strict Base64 decoding then converts that payload to UTF-8. The saved QR image and recorded Base64 string preserve the latter stages even though the original JPEG archive is missing.

Source files: zebra/writeup.md, zebra/solve.py, zebra/reconstructed-qr.png.

6.11 what-did-you-compress

Category: Misc / custom compression

The challenge supplied a known plaintext/compressed pair and `flag.lzw`. The sample bulletin describes a Fisher-Yates symbol shuffle, a xorshift-family generator, low-bit-first packing, wider initial codes and index key `0745B6`. These clues constrain the codec enough to test against exact known plaintext.

The supplied decoder uses these parameters:

Parameter	Reconstructed value
Literal alphabet	Shuffled byte values at codes 0-255
Reset / end	256 / 257
First dynamic entry	258
Initial width	10 bits
Bit order	Least significant bit first

Parameter	Reconstructed value
Width growth	When next index reaches `2**width`
PRNG	xorshift32, shifts 13 left, 17 right, 5 left
Shuffle	Descending Fisher-Yates, swap index `x % (i + 1)`
Seed	`0x0745B6`

The code generates the shuffled alphabet, uses LZW's previous-entry/first-byte rule to extend the dictionary, and compares decoded sample bytes with `sample.txt`. Running the archived script locally confirmed `EXACT SAMPLE MATCH True` for all 2323 bytes. Decoding the supplied flag stream with the same seed returned:

```
m3ss3d_up_compXsion
```

The description explicitly requests a `cruXipher{...}` wrapper, giving:

```
cruXipher{m3ss3d_up_compXsion}
```

The uppercase `X` is present in the decoded bytes and is retained. From the working directory:

```
python what-did-you-compress/solve.py
```

The bulletin claims each tape gets a different table and says the first slots hold control codes. The working solver instead uses literal slots 0-255 and reuses the sample seed successfully. Exact sample recovery supports the implemented parameters; the bulletin wording is not silently substituted for the observed codec. Also, the reset branch only clears previous-entry state, not the whole dictionary. The supplied streams decode, but this exploratory script should not be described as a general decoder for arbitrary reset-containing streams.

Reconstructing the codec

1. Use the known plaintext as the test oracle

The goal is not merely to obtain readable output. `sample.lzw` must decode to every byte of `sample.txt`, including whitespace and punctuation. That exact comparison distinguishes the correct parameters from a decoder that happens to produce a plausible prefix.

2. Rebuild the shuffled literal alphabet

Start with `arr = list(range(256))` and seed `x` with `0x0745B6`. Walk the array from index 255 down to 1. Before each swap, update xorshift32 and select `j = x % (i + 1)`:

```
x ^= (x << 13) & 0xffffffff
x ^= x >> 17
x ^= (x << 5) & 0xffffffff
x &= 0xffffffff
j = x % (i + 1)
arr[i], arr[j] = arr[j], arr[i]
```

The masks retain the intended 32-bit state despite Python's unbounded integers. Each literal code `i` now represents byte `arr[i]`.

3. Read packed codes from the low end

The decoder interprets the byte stream as a little-endian integer and extracts a width-bit code with a right shift and mask. Width starts at 10, not the nine bits used by many LZW examples. The first dynamic dictionary slot is 258 because 256 and 257 are control codes.

4. Extend the dictionary in the correct order

For each data code, emit its dictionary entry. If a previous entry exists, append `previous + current[:1]` at the next free index. At the special next-entry case, the entry is `previous + previous[:1]`. Increase width when the next available index reaches `1 << width`. The exploratory script tests two growth conventions; only the matching convention reproduces the sample exactly.

5. Apply the validated configuration to the flag stream

The complete sample match establishes the shuffle, packing and dictionary rules together. The flag's leading and trailing codes are 256 and 257, and the same seed gives `m3ss3d_up_compXsion` between them. Add the event wrapper specified in `description.txt`, preserving uppercase `X`. No extra guessed transformation is needed after decompression.

Source files: what-did-you-compress/writeup.md, what-did-you-compress/solve.py, what-did-you-compress/description.txt, what-did-you-compress/sample.txt, what-did-you-compress/sample.lzw, what-did-you-compress/flag.lzw.

6.12 puffer

Category: Misc / hardware modeling
Points: 400

AuthStorm advertised a device with no stored keys and a cloud service that shredded its data. `robots.txt` pointed to `/docs.md`; the documented endpoints were `/generate`, `/challenge` and `/response`. The device schematic suggested a 64-stage arbiter physical unclonable function (PUF).

The 39 handout strips were character columns, each with 50000 lines. Constant columns revealed the `0x..., 0x...` format. Transposing the strips reconstructed 50000 challenge/response pairs, with omitted leading zeroes explaining variable hexadecimal widths.

An arbiter PUF is approximately linear in cumulative-product features. The supplied solver reads challenge bits most-significant first, adds a bias column and constructs the feature matrix:

```
s = 1.0 - 2.0 * challenge_bits.astype(np.float64)
phi = np.ones((len(s), 65), dtype=np.float64)
phi[:, 1:] = np.cumprod(s[:, ::-1], axis=1)[:, ::-1]
```

It fits 64 least-squares models against 0/1 response bits and thresholds predictions at 0.5. The writeup reports approximately 99.1% training and 98.9% held-out per-bit accuracy; the archived script trains on all pairs and does not preserve that held-out experiment. Residual errors are consistent with noise, but their physical cause is not established by the supplied files.

The service's proof of work requires a decimal nonce such that SHA-256 of `pow_challenge + str(nonce)` starts with four zero hex digits. After `/challenge` returns a new token, the model predicts the 64-bit response and posts it as a zero-padded `0x%016x` string.

The writeup records two successful responses, including challenge `0xf99f5eb3c216fcf`, prediction `0xe5c26ca97b333aec`, and:

```
cruXipher{puFs_ar3_s0o0_co00o0l!}
```

The other recorded challenge/response pair is `0x161166cb7f46d5bd -> 0xf34aa401966816f0`. The solver expects `strips/1.txt` through `strips/39.txt` beside itself; those inputs are missing. Per-bit accuracy is also different from full 64-bit success probability, so the reported percentage alone is not proof of acceptance. The preserved service results are the evidence for the successful answers.

Modeling and answering the PUF

1. Recover the API and identify the hardware model

The landing page's memory-chip clue did not imply a conventional stored password. `robots.txt` exposed `/docs.md`, and its linked schematic showed a switched delay-line chain ending in an arbiter. A challenge controls the switching paths; arrival order determines a response bit. This explains why the supplied challenge/response observations can train a model.

2. Transpose the shredded observations

Read `strips/1.txt` through `strips/39.txt` as columns and join row `r` from each column. Constant `0`, `x`, comma and space columns identify the original row syntax. Parse both hexadecimal fields as 64-bit unsigned values, preserving leading zeros through numeric conversion rather than expecting fixed-width text.

3. Generate features and fit each response bit

`bits64` expands challenges most-significant bit first. Convert each bit to `1 - 2*c`, calculate suffix products, and prepend the constant bias. This creates a 50000 x 65 feature matrix. Each of the 64 response bits has its own weight vector, fitted against its 0/1 labels. Using the wrong product direction was reported to give approximately chance-level accuracy, supporting the chosen bit order.

4. Solve the proof of work before asking for a challenge

`GET /generate` supplies a token whose payload names a challenge string and difficulty. The solver tries decimal nonces until SHA-256 of the concatenated string begins with the required zero hex digits. It sends the original token and nonce to `/challenge`; decoding the new token reveals the 64-bit device challenge.

5. Predict and serialize all 64 bits

The matrix product gives one score per output bit. Scores above 0.5 become one, and the script reassembles them using `1 << (63 - b)`. The final request body is:

```
{"token": <challenge token>, "response": "0x%016x" % response}
```

The `0x` prefix and sixteen hexadecimal positions are part of the recorded API format. The service returned `Response Verified!` for both preserved examples. This is stronger evidence than per-bit accuracy alone, because the live response must satisfy the service as a complete 64-bit value.

Source files: puffer_writeup.md, solve_puffer.py.

6.13 Give me your seed

Category: Misc / Minecraft seed search
Points: 300

The challenge restricted a Minecraft Java Edition 1.21.3 world seed to 100000-150000 and supplied five screenshots with F3 overlays. Those overlays give coordinates and biomes; visible structures supply additional constraints.

Screenshot	Block coordinates X, Y, Z	Biome / clue
image.png	-485, 153, -52	dark_forest
image2.png	2922, 110, -4678	dark_forest / mansion
image3.png	-1959, 34, 36	desert / portal room
image4.png	-1916, 69, 81	desert / pyramid
image5.png	-4372, 77, 31	jungle / temple

The corrected writeup stresses coordinate signs and flooring: mansion Z `-4677.422` maps to block `-4678`, while portal Z is positive `36.606`. An earlier candidate, 108518, was reported incorrect after coordinate misreadings; it is superseded by the corrected search.

The C solver uses Cubiomes with `MC_1_21_3`, iterating all 50001 seeds. For each it checks a biome-valid mansion within 150 blocks of `(2922,-4677)`, a pyramid within 80 of `(-1916,81)`, and a jungle temple within 80 of `(-4372,31)`. It then prints biomes and stronghold starts for surviving candidates. The tolerances allow for the player's distance from a generation origin.

The recorded unique survivor is 123321:

```
SEED 123321 mansion 2912,-4624 desert -1920,80 jungle -4368,32
BIOMES 29 29 2 2 21 expected 29 29 2 2 21
stronghold -1964,4
stronghold 964,-1452
stronghold 1076,1588
```

A stronghold start need not coincide with its portal room. Cubiomes checks structure placement and biome eligibility here, rather than reconstructing every block. The recorded flag is:

```
cruXipher{123321}
```

The source includes a GCC build command linking Cubiomes' noise, biome, layer, generator, finder, utility and quadbase files. The library is not bundled or pinned, so the search was not rerun in this audit. The five retained screenshots are reproduced in Appendix B. The key lesson is that coordinate precision and version selection are prerequisites for a meaningful seed search.

Constraining and testing the seed range

1. Transcribe the F3 overlays precisely

Record the Minecraft version, block position and biome from each screenshot before coding the search. Negative positions must be floored: `-484.956` belongs to block `-485`. Structure searches use X and Z; the biome checks also retain the player's Y coordinate. The corrected mansion Z is near -4677, not -467, and the portal room's Z is positive.

2. Match the generation version

The solver defines `MC` as `MC_1_21_3`, initializes a generator and applies each candidate seed to the Overworld. This avoids treating screenshots from one release as constraints for another release's generation rules.

3. Filter using three independent structures

For a target structure, `near()` obtains its region configuration and searches the neighboring three-by-three region window. `getStructurePos()` produces the candidate origin; `isViableStructurePos()` checks biome eligibility. Squared distances enforce the mansion, pyramid and temple tolerances without a square-root calculation. A seed is discarded immediately if any one filter fails.

4. Validate the survivor with unused observations

The structure filters leave seed 123321. The next checks compare all five screenshot biomes and enumerate the first three stronghold starts. These checks matter because a nearby structure alone could match by chance. The stronghold start near the portal-room coordinates adds support without requiring the starting staircase and portal room to occupy the same position.

5. Build and run the retained C solver

The original reproduction instructions use Cubiomes and GCC. From the challenge folder, with the required Cubiomes source files available:

```
gcc -O3 -fwrapv -Icubiomes solve.c cubiomes/noise.c \
  cubiomes/biomes.c cubiomes/layers.c cubiomes/biomenoise.c \
  cubiomes/generator.c cubiomes/finders.c cubiomes/util.c \
  cubiomes/quadbase.c -lm -o solve.exe
```

The library revision must expose `MC_1_21_3`; it was not pinned in the source archive. The retained output records one survivor and matching biome IDs. This procedure verifies placement and biome constraints, not a full visual regeneration of every screenshot block.

Source files: Give me your seed/README.md, Give me your seed/solve.c, Give me your seed/images/image.png, Give me your seed/images/image2.png, Give me your seed/images/image3.png, Give me your seed/images/image4.png, Give me your seed/images/image5.png.

Reverse engineering

6.14 Find the PIN

Category: Reverse engineering
Points: 348

The challenge supplied `pocketflag`, a stripped Linux x86-64 ELF accepting a four-character PIN and producing a decoded output. Mixed Boolean-arithmetic expressions obscure the input constraints, while a SIGTRAP-driven virtual machine transforms the encrypted data.

The input checks in main at `0x401713` reduce using `(a | b) - (a & b) = a ^ b` and the corresponding addition identity. With ASCII input bytes `a,b,c,d`, the resulting constraints are:

```
a + d = 0x68
b ^ c = 3
a ^ b = 1
c = 0x34
```

Thus `c='4'`, `b=0x34 ^ 3='7'`, `a=0x37 ^ 1='6'`, and `d=0x68-0x36='2'`. The PIN is `6742`; all four equations were checked locally.

The disassembly identifies a SIGTRAP handler at `0x401176`, encrypted bytes at `0x404040`, and custom bytecode at `0x401958`:

```
a1 17 cc b4 05 cc f0 07
```

`A1` applies an input/index-derived XOR, `B4` applies subtraction, and `F0` advances the output index and loops. Adjusting the saved instruction pointer lets the handler interpret operands around `int3` instructions. The writeup also records `/proc/self/status`, breakpoint-byte and timing checks.

The original environment lacked the binary's Nix-store interpreter. The recorded workaround used an installed Linux dynamic loader:

```
/lib64/ld-linux-x86-64.so.2 /mnt/c/Users/eswar/Downloads/pocketflag 6742
```

Its observed output was `cruXipher{0bfusc4t1on_g0D}`. The literal flag has 26 characters, consistent with the documented output length. This path describes the original Linux environment; it is not a native PowerShell command. The useful reverse-engineering approach is to simplify the constraints before emulating the signal-driven decoder.

Following the input checks and trap handler

1. Separate the PIN constraints from the decoder

The executable's obfuscation makes the arithmetic look more complicated than the checks it enforces. Simplifying the mixed Boolean-arithmetic expressions first avoids spending time emulating a decoder under a wrong input. All four variables are ASCII character values, not integers representing the digits themselves.

2. Solve in dependency order

Start with `c = 0x34`, which fixes the third character as `4`. The XOR relation fixes `b = 0x37`; the second XOR then fixes `a = 0x36`. Finally use `a + d = 0x68` to obtain `d = 0x32`. Replacing those bytes with decimal digit values would fail the sum and XOR constraints.

3. Recognize int3 as part of the virtual machine

The SIGTRAP handler reads bytes after trap instructions and changes the saved instruction pointer. This makes the byte sequence part of an interpreter rather than ordinary straight-line x86 code. Follow the handler's treatment of the `A1`, `B4` and `F0` cases: transform the current encrypted byte, update state, and repeat until 26 output bytes have been processed.

4. Run under the correct loader

The stored interpreter path referenced an unavailable Nix installation. Invoking the binary through `/lib64/ld-linux-x86-64.so.2` bypassed that path in the recorded Linux environment. The verified input is `6742`; the observed 26-character output has the expected event prefix and closing brace. The original binary is not retained, so the handler analysis and loader invocation are documented from the preserved disassembly notes rather than rerun here.

Source files: Find the PIN/writeup.md.

6.15 Paper CPU

Category: Reverse engineering

The challenge asked us to infer an 8-bit CPU from example binaries and input/output pairs, run `challenge.bin` on input `0x2a`, and format `RAM[0x40..0x43]` as eight lowercase hex digits. The inputs comprised `challenge.bin` and four example binaries.

Instructions are documented as opcode/operand byte pairs with a one-byte `0x70` HALT. Low opcode bits select one of four registers. The recovered groups were:

Group	Recorded semantics
0x0r	Load immediate
0x1r	Add register selected by operand low bits
0x2r	XOR immediate
0x3r	Ambiguous register operation
0x4r	Conditional branch with signed byte displacement
0x5r / 0x6r	Store to / load from RAM
0x70	Halt

Branches use byte displacements relative to the position after the operand. The preserved example checks include `kront: 0x2a -> 0x4a`, `vexx: 0x01 -> 0xc3`, `blip: 0x2a -> 0x7e`, and `snok: 0xff -> 0x55`.

Group `0x3r` does not occur in the examples. MOV, ADD, OR and XOR candidates reportedly converge on the same target output for this input. Two decrement-loop blocks bring R0 to zero; the final bytes are `44 56 e6 80`. The first byte agrees with `0x2a ^ 0x6e = 0x44`.

```
cruXipher{4456e680}
```

The notes quote a correct response for CTFd challenge 10. They also say SUB/MUL variants were rejected by the submission oracle, then say only the successful candidate was submitted. Those statements cannot both establish rejected platform attempts. This report retains the successful response and treats other-op rejection as unresolved documentation. A correct answer for one input does not prove a unique ISA.

Deriving and checking the instruction set

1. Identify instruction boundaries

The original workflow compared `file` output and hex dumps for all binaries. `challenge.bin` has 60 two-byte instructions followed by a single-byte HALT, giving 121 bytes. The consistent pair structure separates operands from opcodes and prevents branch targets from being interpreted as instruction numbers.

2. Use the examples as constraints

The low two opcode bits select a register, while the high nibble determines the operation family. Try candidate semantics against every supplied input/output pair before running the target. Load-immediate, register addition, immediate XOR, RAM load/store and branching can then be checked together rather than accepted from one convenient example.

3. Handle signed byte displacements

Branch operands are signed eight-bit offsets relative to the position after the operand. A byte such as `0xea` therefore represents a negative displacement, not a large forward jump. The recorded loop-backs `40 ea` and `40 ec` return to instructions 13 and 33. Interpreting these offsets in instruction units instead of byte units would send execution to different locations.

4. Keep the unobserved opcode family explicit

The examples do not constrain group `0x3r`. MOV, ADD, OR and XOR interpretations converge on the same four output bytes for input `0x2a`; that convergence supports the requested answer without uniquely identifying the operation. The source's contradictory claim about rejected submissions is not needed to establish the accepted output.

5. Read the requested memory window in order

The documented emulator starts with zeroed RAM and R1/R2, runs the two decrement loops, and reads addresses 0x40, 0x41, 0x42 and 0x43. Concatenate `44`, `56`, `e6` and `80` as eight lowercase hex digits. This is a byte sequence, not a host-endian integer conversion, so reversing the bytes would change the answer.

Source files: combined-writings.md.

6.16 T2F Program

Category: Reverse engineering
Points: 344

`t2f-masterclass.zip` advertised a single 131072-byte `flag.txt`, but standard ZIP tools failed to inflate its method-8 stream. The recorded archive size was 121646 bytes, compressed entry size 121532, and CRC32 `0xcce1c38d`. Zlib reported `invalid literal/lengths set`, suggesting a code-tree error before literal decoding.

Parsing the compressed header revealed one dynamic-Huffman DEFLATE block with `BFINAL=1`, `BTYPE=2`, `HLIT=257`, `HDIST=2`, and `HCLEN=19`. Its literal-code Kraft sum was `1.00390625`: oversubscribed by `2 * 2**-9`, exactly two extra nine-bit code contributions.

That observation narrows repair to removing two nine-bit symbols. The documented solver tested all `C(46,2)=1035` pairs. Setting code lengths for symbols 100 (`d`) and 113 (`q`) to zero uniquely yielded the expected 131072 bytes and stored CRC32. The resulting Kraft sum was 1.0.

```
[+] after repair: Kraft sum = 1.0
[+] inflated 131072 bytes, CRC32 = 0xcce1c38d, match = True
```

The extracted plaintext reportedly contained the flag at byte offset 65525:

```
cruXipher{sighunzips}
```

The recorded response for CTFd challenge 11 reports `correct`. The search's unique length-and-CRC match is strong evidence among the tested repairs; CRC32 alone is not a general cryptographic proof of file identity. Since neither archive nor solver is supplied, those repair results are recorded evidence rather than a new reproduction. This challenge rewards diagnosing the compressed bitstream instead of repeatedly switching archive extractors.

Repairing the malformed DEFLATE tree

1. Confirm that the failure is inside the compressed stream

```
file t2f-masterclass.zip
zipinfo -v t2f-masterclass.zip
unzip -o t2f-masterclass.zip -d unz
7z x t2f-masterclass.zip
```

The directory metadata identifies one ordinary deflated entry, but both extractors fail with a data error. Python's `zlib.decompress(data, -15)` reports `invalid literal/lengths set`. The negative window parameter selects a raw DEFLATE stream, so this error points to its Huffman lengths rather than a missing zlib wrapper.

2. Check the code-length budget

For codeword lengths `l`, a complete binary prefix tree has Kraft sum `sum(2**-l) = 1`. The decoded literal-length table gives 1.00390625, an excess of 0.00390625. Since `2 * 2**-9` equals that excess, deleting two nine-bit contributions is a focused repair hypothesis rather than an arbitrary bit flip.

3. Enumerate the narrow repair space

There are 46 nine-bit candidates and 1035 unordered pairs. For each pair, remove those two symbol lengths, rebuild the candidate decoding table, and attempt to recover the entry. Reject candidates that fail decoding, have the wrong length, or disagree with the stored CRC32. The recorded unique survivor removes symbols 100 and 113.

4. Check output before extracting the flag

The accepted repair emits exactly 131072 bytes with CRC32 `0xcce1c38d`. Only after those checks does a substring search locate the flag:

```
grep -abo 'cruXipher{[^}]*}' /tmp/opencode_t2f/flag.txt
```

The recorded result begins at offset 65525. No second encoding layer is required; the flag is literal text in the recovered entry. The repair script and ZIP are absent, so these steps describe the documented search and its checks rather than supplying a replacement bitstream parser.

Source files: combined-writings.md.

Pwn

6.17 mlg

Category: Pwn
Points: 400

The target was `134.209.144.148:7834`, with a stripped x86-64 PIE executable named `chall`. The writeup records NX, full RELRO and no stack canary. It also lists SHSTK/IBT properties; those ELF properties alone do not demonstrate runtime enforcement, which matters when assessing the documented return-address attack.

Leak and hidden overflow

Menu option 2 leads through the doge dialogue. Answering `no` to the love question and `yes` to the kill question prints a function pointer after `AAAAAAAAAAAA`. The archived script subtracts `0x1387` to recover the PIE base. This is a legitimate `%p` argument, rather than an uncontrolled format-string vulnerability.

Hidden menu input 420 invokes an `0x80`-byte read into a `0x20`-byte stack buffer. With no canary, 32 bytes reach saved RBP and the next eight bytes reach the saved return address.

The normal win function at `0x158c` checks a quickscope counter. Entering at `0x15a7` bypasses that gate and starts the `fopen("flag.txt", "r")` path. Since this internal entry uses RBP-relative locals without running a prologue, the exploit supplies a writable fake frame in the mapped data area.

```
base = int(re.search(rb'AAAAAAAAAAAA(0x[0-9a-f]+)', data).group(1), 16) - DOGE_LEAK
p.send(b'A' * 0x20 + p64(base + SCRATCH) + p64(base + WIN_BODY))
```

The supplied constants are `SCRATCH=0x4200` and `WIN_BODY=0x15a7`. The fake frame leaves room for both the flag buffer at `rbp-0x70` and the file pointer at `rbp-8`. The source explains stack alignment at the internal entry and notes that the successful path calls `exit`, so a complete caller frame is unnecessary.

The recorded remote output contains:

```
cruXipher{mIg_n0_$C0p3_G3t_rek7_m8}
```

The second character of `mIg` is uppercase `I`, preserved exactly. The standalone `solve.py` matches the documented leak and payload; it was inspected without opening a network connection. A local run also needs `flag.txt`, which is absent. The exploit combines a deliberate PIE disclosure, an unbounded read and careful frame placement.

Exploit walkthrough

1. Map the menu and protections

```
file chall
checksec --file=chall
```

PIE requires a runtime code address, NX prevents executing injected stack bytes, and full RELRO rules out the obvious GOT overwrite. The missing stack canary makes the hidden oversized read the direct route to controlling a return address.

2. Obtain the runtime code address through option 2

The retained solver waits for `otion:`, chooses `2`, answers `no` to the doge-love prompt and `yes` to `kil doge?`. It extracts the hexadecimal pointer following `AAAAAAAAAAAA` and subtracts `DOGE_LEAK = 0x1387`. The recorded example gives base `0x55fa356bc000` from leak `0x55fa356bd387`.

3. Choose an internal win entry that avoids the guard

The function begins at offset `0x158c`, but its counter gate prevents the ordinary route. `WIN_BODY = 0x15a7` starts at the flag-opening path after that check. Entering inside a function requires accounting for the frame setup that was skipped; otherwise its RBP-relative accesses would use an uncontrolled address.

4. Supply a writable fake frame

The payload uses `base + 0x4200` as RBP. The flag buffer is then at `base + 0x4190`, and the file pointer slot at `base + 0x41f8`, avoiding overlap. The successful path exits the process after printing, so it does not need a normal caller frame to return to.

Payload field	Length	Purpose
A padding	32 bytes	Fill the local buffer
Packed fake RBP	8 bytes	Provide writable local-variable addresses
Packed return address	8 bytes	Enter `base + 0x15a7`

5. Trigger option 420 and read the result

The script chooses `420`, waits for `gamertag:`, and sends the 48-byte payload. The vulnerable read permits up to 128 bytes, so this payload reaches the saved return slot without needing the full limit. On return, the internal win path opens `flag.txt`, reads it, prints it and calls `exit`. The archived remote output is the result evidence; no remote exploit was rerun while preparing this report.

Source files: mlg_420blazeit_writeup.md, chall, solve.py.

OSINT

6.18 santas-nice-list

Category: OSINT
Points: 500

The challenge supplies 41 addresses and emphasizes delivering gifts at an exact midnight time. This suggests examining the relationship between location, longitude and time. Several listed businesses lie close to particular longitude meridians:

Location	Approximate longitude	Mean-solar time offset
Old Arabi Food Store	-90 degrees	-6 hours
ASR Food Market, Philadelphia	-75 degrees	-5 hours
Kaka Kariyana, Sirsa	+75 degrees	+5 hours
AK & Co., Anchorage	-150 degrees	-10 hours
Ernie's Coffee Shop	-120 degrees	-8 hours
Takano cafe, Shiraz	+52.5 degrees	+3 hours 30 minutes
Roadhaven Cafe, Marulan	+150 degrees	+10 hours

Earth rotates through 360 degrees in 24 hours, giving 15 degrees per hour. An east-positive longitude therefore gives an approximate mean-solar time offset:

```
offset = longitude / 15
UTC time of mean-solar midnight = (-offset) mod 24
```

These offsets differ from official civil time zones. Sirsa, for example, is near 75 degrees east, corresponding to approximately UTC+5 in mean-solar time, while Indian Standard Time is UTC+5:30. The longitude observations support the geographic interpretation, but do not establish the location-to-character extraction.

Recognizing the encoded sequence

The intermediate sequence associated with the 41-location list is:

```
MNZHKWDJOBUGK4T3KMYFEUZGRXDA4TUNBJXINDSPU
```

Its uppercase letters and digits 2 through 7 match the standard Base32 alphabet. The original sequence has 41 characters; an unpadded standard Base32 message cannot have a length congruent to 1 modulo 8. A character is therefore missing if this is intended to be a complete Base32 message.

Inserting `S` at one-based position 23 produces a 42-character sequence:

```
MNZHKWDJOBUGK4T3KMYFEUSZGRXDA4TUNBJXINDSPU
```

The insertion reconstructs a missing character. Its justification from the location list remains unresolved, so this step is not presented as a proven geographic derivation.

Decoding and verification

In CyberChef, applying **From Base32** to the corrected sequence yields the flag. Python reproduces the same result:

```
import base64
```

```
encoded = "MNZHKWDJOBUGK4T3KMYFEUSZGRXDA4TUNBJXINDSPU"
padded = encoded + "=" * (-len(encoded) % 8)
decoded = base64.b32decode(padded).decode("utf-8")
print(decoded)
assert base64.b32encode(decoded.encode()).decode().rstrip("=") == encoded
```

Output:

```
cruXipher{S0RRY4n0rthSt4r}
```

Reverse encoding reproduces the corrected sequence exactly, confirming the decoding. It does not prove how the original addresses yield the intermediate sequence or establish platform acceptance. The final decode is reproducible; the geographic extraction remains the incomplete part of the solution.

Source files: santas-nice-list-writeup.md.

6.19 Is This Really OSINT?

Category: OSINT / web / cryptography

The application at `https://is-this-really-osint.cruxipher.crux-bphc.com` combined six location checks, a shortlink clue, an encrypted archive and a final cipher. Completing the six `POST /api/verify` checks produced an `/api/status` response with all stages complete.

Stage	Recorded coordinates	Location / clue
1	47.4580, 19.0838	Budapest, OBI / B
2	-23.887115, 32.048528	Massingir Dam, Landsat 3 / 3
3	40.664111, 47.110056	Lake Guakhmaz, Landsat A / A
4	36.18486, -5.43664	Palmones / Los Barrios, Conforama / C
5	40.7607722, -111.8675192	Salt Lake City, Salon O / O
6	41.885834, -87.624736	Chicago, East Lake St / Nando's / N

These are recorded stage answers, not newly geolocated photographs. The sequence `b3acOn` identifies `https://tinyurl.com/b3acOn`, recorded as redirecting to `https://github.com/randombsgo234-pixel/ubiquitous-garbanzo`. The completed session's `/api/key` returned the archive password `th3-m4p-1s-n0t-th3-t3rr1t0ry-F4wCKY`. This is a puzzle key used to open the challenge artifact; session cookies remain redacted.

The repository contained `notes.7z`; opening it with the returned password exposed `LAST AND FINAL I PROMISE.txt`. That note contained:

```
drwL{50z3u1m35_vv3_sm4g_y45_hu3_051o7_w3_f1r_4y0og_vv3_j4z}
```

The Vigenere key `BACON` comes from the stage clue after removing `3`. Decryption preserves case, leaves digits and punctuation untouched, and advances the key only on letters. The result is:

```
cruX{50m3t1m35_th3_fl4g_w45_th3_051n7_w3_d1d_4l0ng_th3_w4y}
```

The short `cruX` prefix is supported by the notes' correct response for challenge 13 and a later `already_solved` response. Decrypting and re-encrypting the preserved ciphertext confirms the exact result. The website, shortlink and repository were not revisited; current contents are not asserted. This chain demonstrates why an application key, archive password and final cipher key must be tracked as separate intermediate artifacts.

Following the complete clue chain

1. Finish the six location checks

Use the stage coordinates listed above with `POST /api/verify`. The preserved record establishes the accepted coordinates and clue letters, but does not preserve the original photos or a complete account of locating each scene. After the sixth stage, `/api/status` reported:

```
{"completedStages":6,"totalStages":6,"completed":true}
```

This completed state is needed before requesting the application key. The six clues combine as `b3acOn`, whose case is retained for the shortlink.

2. Retrieve the archive key from the completed session

The original request included a session cookie. Its value is omitted here, but the endpoint and the returned challenge-derived password are retained. This value opens the archive; treating it as a final flag skips a later required layer.

3. Follow the shortlink to the repository

```
curl -sS -i -L https://tinyurl.com/b3acOn
```

The recorded redirect points to `randombsgo234-pixel/ubiquitous-garbanzo` on GitHub. Its `notes.7z` is the encrypted artifact. The repository and redirect are historical evidence, not assertions about their present contents.

4. Open the archive and preserve the ciphertext

The original 7-Zip extraction used password `th3-m4p-1s-n0t-th3-t3rr1t0ry-F4wCKY`. The extracted `LAST AND FINAL I PROMISE.txt` contains a flag-shaped ciphertext beginning `drwL{`. Since the prefix itself is encrypted, wrapping the ciphertext in a new event prefix would not solve the puzzle.

5. Apply the separate Vigenere key

Remove `3` from `B3ACON` to obtain `BACON`. Subtract its repeated letter shifts only for alphabetic characters, preserving the original case and leaving numbers, underscores and braces untouched. The first four ciphertext letters `drwL` become `cruX`; the key position continues across the rest of the message rather than restarting inside braces.

6. Verify the plaintext and retain the short prefix

Re-encrypting the recovered plaintext with `BACON` reproduces the note exactly. The preserved CTFd responses are `correct` followed by `already_solved`. Together these support the exact short-prefix flag; expanding it to `cruXipher` would alter the accepted string.

Source files: combined-writings.md.

7. Cross-Challenge Technical Observations

Several solutions depend on distinguishing a visible result from the channel that actually carries evidence. Boxy proxy returns an HTTP error while delivering its message over TCP. Wait For It puts data in traffic rhythm rather than IRC text. PDF Prince separates extracted text, displayed glyphs and historical streams. JYU requires editor state before keyboard substitution. In each case, selecting the correct representation matters before choosing a decoding tool.

Independent constraints make reconstructions more credible. The compression sample provides a byte-exact oracle; T2F uses expected length and CRC; the Minecraft search combines separate structures and biome readings; Internal Affairs matches the salary increase against the company-average jump. Each check has a boundary: CRC is not cryptographic authenticity, a generated structure origin is not a block-for-block world reconstruction, and a final encoding round trip does not prove the extraction that preceded it.

The exploitation records show different forms of misplaced trust. GraphQL filters reveal restricted values, a certificate verifier accepts an identity backed by a recoverable key, and an internal function entry bypasses a win condition. These are related by missing validation across a boundary, not by a single universal exploit technique.

8. Lessons Learned

1. Preserve original attachments alongside solvers. A screenshot, transcript excerpt or final flag file does not replace a WAV, PCAP, archive, executable or exact library build. Most narratives here cannot be replayed end to end from this folder.

2. Save machine-readable verification evidence. Keep request/response logs with credentials removed, platform acceptance records, decoder outputs and hashes of both input and recovered artifacts. Label a service response separately from a final platform submission.

3. Record intermediate artifacts explicitly. Dropped as a Baby's hidden key, the OSINT archive password and the final Vigenere key serve different roles. Their flag-like appearance can otherwise cause a premature stopping point.

4. Retain uncertainties instead of smoothing them away. An ambiguous opcode can still yield a supported target answer; an unproven place-to-symbol rule cannot become proven through a successful Base32 round trip.

5. Pin dependencies and environment details. Preserve Cubiomes revisions, libc and loader files, compiler flags, bit order, character case and image dimensions. Document hard-coded manual inputs such as the recovered tile arrangement.

6. Validate literal counts and sizes mechanically. The JYU and PIN counts provide useful consistency checks against the recorded byte sequences. Treat addresses and assertions as checkable data.

9. Conclusion

These 19 writeups bring together our work across web security, forensics, reverse engineering, exploitation and OSINT. The solutions rely on careful reading of formats and protocols, small reproducible experiments, and checks against independent clues. The final Base32 decode for santas-nice-list is reproducible, although its geographic extraction remains unresolved.

Appendix A. Source Inventory and Reproduction

The companion challenge inventory maps every challenge to its writeups, helper scripts, retained artifacts and missing evidence. It also lists all 50 original files with sizes and SHA-256 digests. Shared sources are intentionally listed for multiple challenges; this does not create duplicate challenge entries.

Python scripts that interact with services were inspected without execution. Only the offline compression decoder and isolated local functions/arithmetic were exercised. Several supplied scripts write reconstructed images or flag files; rerun them in a copied workspace if preserving the archive unchanged is required. Source code remains available through the links in each challenge section.

The main script dependencies are Pillow/NumPy for image operations, OpenCV for the zebra solver, cryptography/requests for boxy proxy, NumPy/requests for puffer, pwntools for mlg, and Cubiomes plus a C compiler for the Minecraft search. Availability of those packages does not supply the missing challenge inputs.

Appendix B. Preserved Minecraft Screenshots

The five source screenshots below accompany section 6.13. The coordinate table preserves the documented F3 readings; exact version and coordinate signs are essential to the corrected seed search. Full-resolution source links remain available in Markdown.



Figure 3. Forest terrain; documented block position (-485,153,-52).



Figure 4. Woodland mansion; documented block position (2922,110,-4678).



Figure 5. Stronghold portal room; documented block position (-1959,34,36).



Figure 6. Desert pyramid; documented block position (-1916,69,81).



Figure 7. Jungle temple; documented block position (-4372,77,31).

Appendix C. Verification Notes and Glossary

Local checks performed for this report are listed in `report_build/verification_results.json`: original-file hash preservation; exact LZW sample recovery and flag decode; CA-factor multiplication and private-key signing; Vigenere decryption for the two preserved cipher strings; House Music alphabet indexing; Rice Writing's first five area remainders; PIN equations; PDF Prince's archived XOR/glyph substitution; the santas-nice-list Base32 decode and reverse-encoding check; the zebra Base64 payload; JYU's literal lengths and supplied character mapping; ELF identity/selected constants. These checks validate their stated components, not every historical observation in the source narratives.

Term	Meaning in this report
CTFd	Competition platform whose responses are quoted in selected notes
PIE / ASLR	Position-independent executable / address randomization
ROP	Return-oriented programming using existing instruction sequences
GOT / RELRO	Dynamic-linking pointer table / protection of relocation data
PUF / CRP	Physical unclonable function / challenge-response pair
LSB	Least significant bit of a channel value
IDAT / IHDR	PNG compressed pixel-data / image-header chunks
Kraft sum	Sum of codeword contributions used to assess prefix-code lengths
Vigenere	Repeated alphabetic-key substitution used by two layered puzzles

All references supporting the team's solving history are local source files. External services and repositories named in the text are historical leads copied from those sources, not independently consulted references.